Imitation Learning with Dagger

Code Implementation Overview

Sample from trajectory

```
ac = ptu.to_numpy(policy(ptu.from_numpy(ob.reshape(1, -1))).sample())
ac = ac[0]  # Retrieve the action value
next_ob, rew, done, _ = env.step(ac)
steps += 1
```

Collecting Trajectories

```
paths, envsteps_this_batch = utils.sample_trajectories(env, actor,
params['batch_size'], params['ep_len'])
```

Get Expert Policy

```
# Update actions in each path using expert policy
for path in paths:
    path["action"] = expert_policy.get_action(path["observation"])
```

Get Sampled Data

```
ob_batch =
replay_buffer.obs[np.random.permutation(replay_buffer.obs.shape[0])
[:params['batch_size']]]
ac_batch =
replay_buffer.acs[np.random.permutation(replay_buffer.obs.shape[0])
[:params['batch_size']]]
```

Training the Actor

```
# Update the actor model using sampled observations and actions
train_log = actor.update(ob_batch, ac_batch)
```

Behavioral Cloning

Sanity

Ant:

```

```

```
python cs285/scripts/run_hw1.py --expert_policy_file
cs285/policies/experts/Ant.pkl --env_name Ant-v4 --exp_name bc_ant --
n_iter 1 --expert_data cs285/expert_data/expert_data_Ant-v4.pkl --
video_log_freq -1
```

HalfCheetah:

```
python cs285/scripts/run_hw1.py --expert_policy_file
cs285/policies/experts/HalfCheetah.pkl --env_name HalfCheetah-v4 --
exp_name bc_halfcheetah --n_iter 1 --expert_data
cs285/expert_data/expert_data_HalfCheetah-v4.pkl --video_log_freq -1
```

Varying Batch Sizes with Fixed Episode Length:

▶ Details

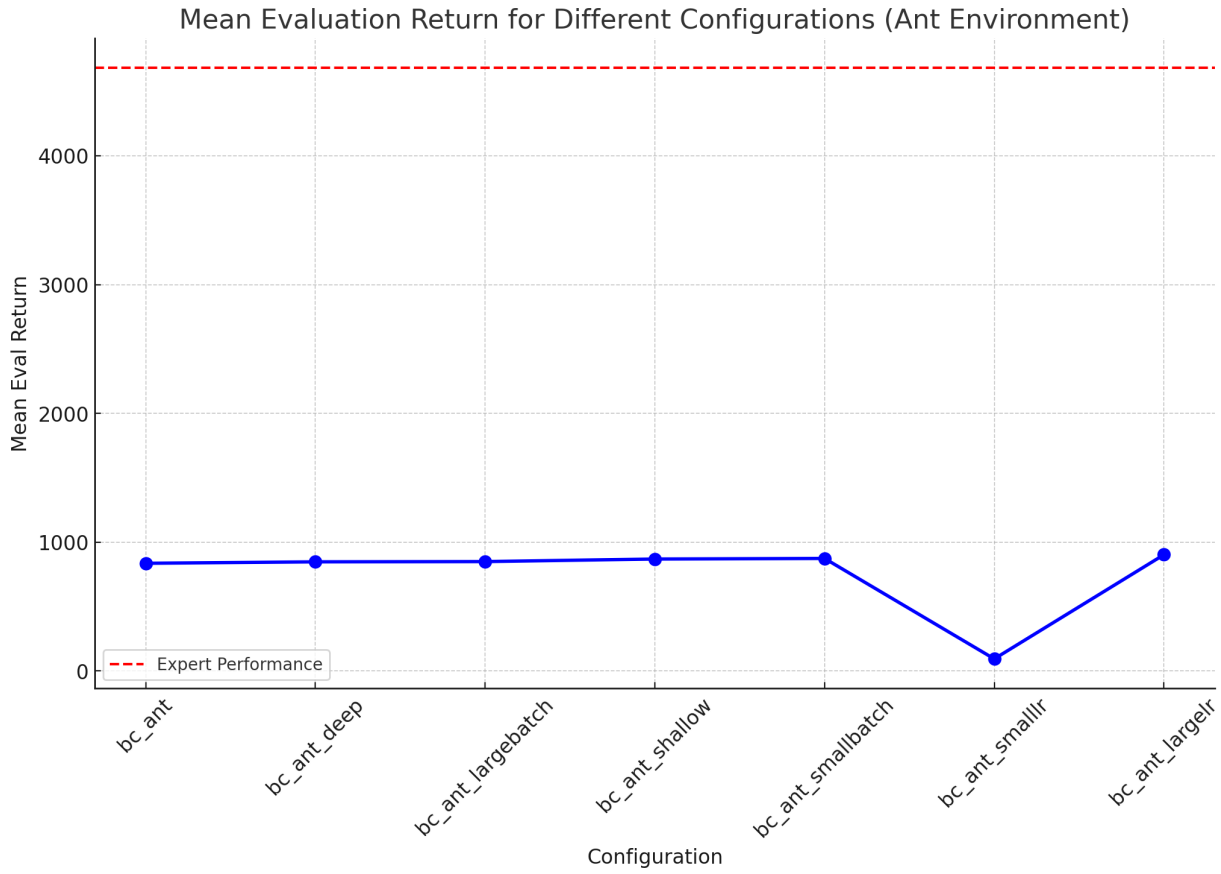
Exploring Network Depth and Width:

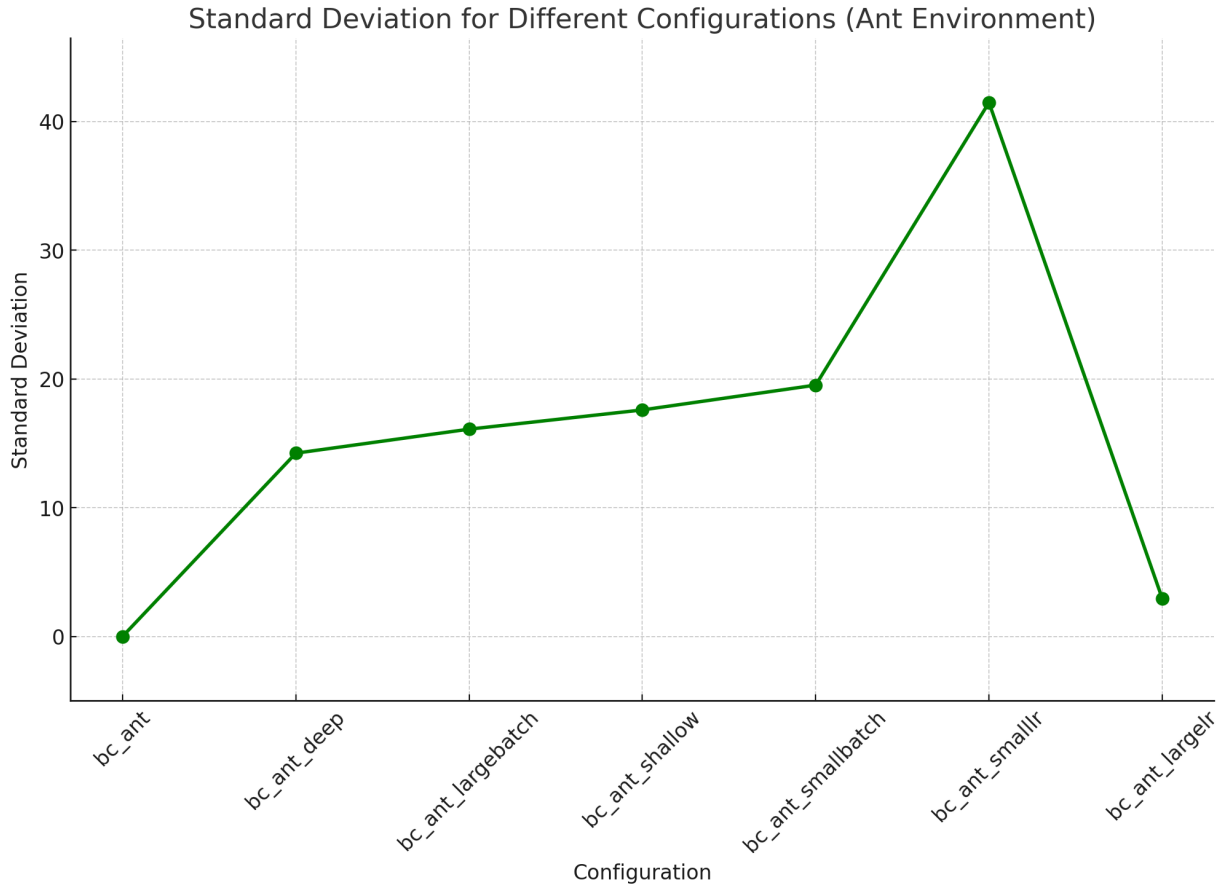
▶ Details

Changing Learning Rates:

▶ Details

Behavioral Cloning Conclusion





bc_ant: Network: Default, Batch: Default, LR: Default

bc_ant_deep: Network: 4 layers, 128 units, Batch: Default, LR: Default

bc_ant_largebatch: Network: Default, Batch: 30000, LR: Default

bc_ant_shallow: Network: 1 layer, 32 units, Batch: Default, LR: Default

bc_ant_smallbatch: Network: Default, Batch: 100, LR: Default

bc_ant_smalllr: Network: Default, Batch: Default, LR: 1e-3

bc_ant_largelr: Network: Default, Batch: Default, LR: 5e-2

Dagger

Add `--do_dagger` to the end of each commands above.

Varying Batch Sizes with Fixed Episode Length:

Exploring Network Depth and Width:

Changing Learning Rates:

Changing Num Iters: